

elements_view needs its own sentinel

Document #: P1994R0
Date: 2019-11-25
Project: Programming Language C++
LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>
Christopher Di Bella
<cjdb.ns@gmail.com>

§ 1 Abstract

`elements_view` needs its own `sentinel` type.

§ 2 Discussion

`elements_view` is effectively a specialized version of `transform_view`; the latter has a custom `sentinel` type, and so should `elements_view`.

More generally, if a range adaptor has a custom iterator, it should probably also have a custom `sentinel`. The rationale is that the underlying range's `sentinel` could encode a generic predicate that is equally meaningful for the adapted range, leading to an ambiguity.

For example, consider:

```
struct S { // sentinel that checks if the second element is zero
    friend bool operator==(input_iterator auto const& i, S) requires /* ... */
    { return get<1>(*i) == 0; }
};

void algo(input_range auto&& r) requires /* ... */ {
    for (auto&& something : subrange{ranges::begin(r), S{}})
    {
        /* do something */
    }
}

using P = pair<pair<char, int>, long>;
vector<P> something = /* ... */;

subrange r{something.begin(), S{}};

algo(r | view::keys); // checks the long, effectively iterating over r
completely
algo(r | view::transform(&P::first)); // checks the int and stops at the first zero
```

algo is trying to use the sentinel `S` to stop at the first element `e` for which `get<1>(e)` is zero, and it works correctly for all ranges of `pair<char, int> ...` except things like `r | view::keys`. In the latter case, `ranges::begin(r | view::keys) == S{}` calls `elements_view::iterator`'s `operator==` instead, which compares the *underlying range*'s iterator against the sentinel. In the above example, this means that we check the `long` instead of the `int`, and effectively iterate over the entire range.

A custom sentinel is needed to avoid this problem. `elements_view` is the only adaptor in the current WP that has a custom iterator but not a custom sentinel.

§ 3 Wording

This wording is relative to [\[N4835\]](#).

[Drafting note: These changes, including the overload set for `end`, match the specification of `transform_view`. This seems appropriate since `elements_view` is just a special kind of `transform`.]

Edit 24.7.16.2 [\[range.elements.view\]](#), class template `elements_view` synopsis, as indicated:

```
namespace std::ranges {
    [...]

    template<input_range R, size_t N>
        requires view<R> && has-tuple-element<range_value_t<R>, N> &&
            has-tuple-element<remove_reference_t<range_reference_t<R>>, N>
    class elements_view : public view_interface<elements_view<R, N>> {
    public:
        [...]

+       constexpr auto end()
+       { return sentinel<false>{ranges::end(base_)}; }
+
+       constexpr auto end() requires common_range<R>
+       { return iterator<false>{ranges::end(base_)}; }
+
+       constexpr auto end() const
+       requires range<const R>
+       { return sentinel<true>{ranges::end(base_)}; }
+
+       constexpr auto end() const
+       requires common_range<const R>
+       { return iterator<true>{ranges::end(base_)}; }

-       constexpr auto end() requires (!simple-view<R>)
-       { return ranges::end(base_); }

-       constexpr auto end() const requires simple-view<R>
-       { return ranges::end(base_); }

        [...]

    private:
        template<bool> struct iterator;                                // exposition only
```

```
+ template<bool> struct sentinel;           // exposition only
    R base_ = R();                         // exposition only
};
}
```

Edit 24.7.16.3 [\[range.elements.iterator\]](#), class template `elements_view<R, N>::iterator` synopsis, as indicated:

```
namespace std::ranges {
    template<class R, size_t N>
    template<bool Const>
    class elements_view<R, N::iterator { // exposition only
    [...]

    public:
        [...]

-     friend constexpr bool operator==(const iterator& x, const
        sentinel_t<base_t>& y);

        [...]

-     friend constexpr difference_type
-     operator-(const iterator<Const>& x, const sentinel_t<base_t>& y)
-     requires sized_sentinel_for<sentinel_t<base_t>,
        iterator_t<base_t>>;
-     friend constexpr difference_type
-     operator-(const sentinel_t<base_t>& x, const iterator<Const>& y)
-     requires sized_sentinel_for<sentinel_t<base_t>,
        iterator_t<base_t>>;
    };
}
```

Delete the specification of these operators in 24.7.16.3 [\[range.elements.iterator\]](#) (p12, 22 and 23):

```
friend constexpr bool operator==(const iterator& x, const
    sentinel_t<base_t>& y);
```

12 ~~Effects: Equivalent to: `return x.current_ == y;`~~

...

```
friend constexpr difference_type
operator-(const iterator<Const>& x, const sentinel_t<base_t>& y)
requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;
```

22 ~~Effects: Equivalent to: `return x.current_ == y;`~~

```
friend constexpr difference_type
operator-(const sentinel_t<base_t>& x, const iterator<Const>& y)
requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;
```

23 ~~Effects: Equivalent to: `return (y - x);`~~

Add a new subclass after 24.7.16.3 [\[range.elements.iterator\]](#):

§ ??? Class template `elements_view::sentinel` [`ranges.element.sentinel`]

```
namespace std::ranges {
    template<class R, size_t N>
    template<bool Const>
    class elements_view<R, N>::sentinel {           // exposition only
    private:
        using base_t = conditional_t<Const, const R, R>; // exposition only
        sentinel_t<base_t> end_ = sentinel_t<base_t>(); // exposition only
    public:
        sentinel() = default;
        constexpr explicit sentinel(sentinel_t<base_t> end);
        constexpr sentinel(sentinel_t<!Const> other)
            requires Const && convertible_to<sentinel_t<R>, sentinel_t<base_t>>;

        constexpr sentinel_t<base_t> base() const;

        friend constexpr bool operator==(const iterator<Const>& x, const sentinel& y);

        friend constexpr range_difference_t<base_t>
            operator-(const iterator<Const>& x, const sentinel& y)
                requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;

        friend constexpr range_difference_t<base_t>
            operator-(const sentinel<Const>& x, const iterator& y)
                requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;
    };
}

constexpr explicit sentinel(sentinel_t<base_t> end);
```

- 1 *Effects:* Initializes `end_` with `end`.

```
constexpr sentinel(sentinel_t<!Const> other)
    requires Const && convertible_to<sentinel_t<R>, sentinel_t<base_t>>;
```

- 2 *Effects:* Initializes `end_` with `std::move(other.end_)`.

```
constexpr sentinel_t<base_t> base() const;
```

- 3 *Effects:* Equivalent to: `return end_;`

```
friend constexpr bool operator==(const iterator<Const>& x, const sentinel&
    y);
```

- 4 *Effects:* Equivalent to: `return x.current_ == y.end_;`

```
friend constexpr range_difference_t<base_t>
    operator-(const iterator<Const>& x, const sentinel& y)
        requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;
```

- 5 *Effects:* Equivalent to: `return x.current_ - y.end_;`

```
friend constexpr range_difference_t<base_t>
    operator-(const sentinel<Const>& x, const iterator& y)
```

```
requires sized_sentinel_for<sentinel_t<base_t>, iterator_t<base_t>>;
```

⁶ *Effects:* Equivalent to: `return x.end_ - y.current_;`

§ 4 References

[N4835] Richard Smith. 2019. Working Draft, Standard for Programming Language C++. <https://wg21.link/n4835>